

Modelling Inland CO₂ Transportation

Paulius Cebataraukas

August 2022

1 Introduction

This summer project is part of the ACCSESS work package 9 project - Pioneering CCUS chains. The problem at hand is how to transport CO₂ from inland sources to ports. In this project we only look at problems with only one port. In particular, given a starting position, we compute the shortest distance to the port using several transportation methods. The methods consisting of only one mode of transportation are via trucks, trains and barges via roads, railroads and rivers, respectively. We have also implemented a pipeline distance. The option to use two different transportation methods has also been implemented, which computes the distance using a combination of two different modes of transportation, to allow more flexibility. The project has consisted of three parts. First, we have used QGIS, a geographical software, to extract and export geographical data. Secondly, we have computed the shortest paths. Lastly, we have compared our results to the ones QGIS can produce using builtin algorithms.

2 Methodology

There have been two main parts to this project. The first part concerned the use of QGIS, a free geographical information system, to process the geographical data and get an output file with all the relevant data. In the second part of this project we have analyzed the data created in the first part. The work in this part was done using Python.

2.1 QGIS

The goal in the QGIS section was as follows: a user must define an area of interest(AOI), which in QGIS is a polygon that covers the area which the user wants to study. Then, we have developed a model in QGIS, which creates a quadratic grid over that region. The gridsize is user defined. For each grid cell, our algorithm will store the information about whether there is a road, railroad or river available in said grid cell. Then this data is exported as a CSV file. For a more detailed information refer to the file about the use of QGIS.

There are several ways to work in QGIS. You can work directly with the GUI and apply functions manually. While this works well, we don't really like manual labor and things that could be automated are nice to automate. Let me give an example: If you want to run 10 functions on your map every time you open QGIS, wouldn't you prefer if those 10 functions could be collected into one function?¹ This can be done in several ways using QGIS. QGIS supports Python, and its API is called PyQGIS.² You can either run simple code directly in the Python console in QGIS, or you can write it in a separate file and then run it. It is also possible to write a python script that then acts as any other QGIS function. As I was struggling with the PyQGIS API in the beginning, I discovered another solution - the graphical modeler. This resembles low-code platforms, as it allows you to create a workflow where you can chain many functions. So whatever you can do manually using the processing toolbox you can also do using the graphical modeler. Hence, if you get a bit familiar with the GUI and know what functions to use in the processing toolbox, then you can automate this process using the graphical modeler. The only thing I haven't been able to do in the graphical modeler is to run loops. This is one of the few cases where I would actually use the Python console instead. Also in the case you want to run operations

¹Yes, you would.

²https://docs.qgis.org/3.22/en/docs/pyqgis_developer_cookbook/index.html

or make objects that are not naturally supported in QGIS processing, then it would also be natural to use the Python console.

2.2 Python script

Once we had the CSV file from QGIS we began working with it in Python. The main packages used in this process were Pandas, Networkx, Numpy and Matplotlib. We imported the CSV file as a Pandas dataframe and most of our computations on this dataset was done using Pandas functionality. Networkx was used to compute shortest paths using Dijkstra. Numpy was used for vectorized calculations, as we know that python for loops are not very fast(read as very slow). Finally, we used Matplotlib to visualise the results of our error analysis.

The Python code produces a new and expanded version of the CSV file from QGIS, which contains the distances from each grid cell to the port. For the distances that use only one mode of transportation, we simply compute the shortest distance in kilometers and add it to the dataset as a new column. For distances using two modes of transportation we return three values: the distance travelled along the first mode of transportation in kilometers, the distance travelled using the second mode of transportation in kilometers, and the id of the transition grid cell where we switched from one mode of transportation to the other. In particular, this means that we only allow to change the transportation method once. So no back and forth here!

2.3 Shortest path

Let's next examine how we define our graph and compute the shortest distances. Let us begin by describing the easiest method first, the pipeline. We make the assumption that we can build the pipeline in a straight line from each grid cell to the port. In reality, this will most likely not be possible, thus a tortuosity multiplier should be applied to get a more realistic distance. In our dataset we have indexed each gridcell by (x, y) . These grid cell coordinates have no dimension, so to compute a length we will need to multiply by the size of our grid. The size of our grid is the length and width of the grid cells. So the pipeline distance from some point, say Point 1 with coordinates (x_1, y_1) , to the port, with coordinates (x_2, y_2) , is given by

$$distance = gridsizes \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}.$$

Now moving on to the transportation methods where we need to implement a graph and use a shortest path algorithm. We have three such

graphs, one for each of the following methods: road, rail, river. Each of the graphs are undirected and weighted. Without loss of generality let us examine the graph for the road system. The nodes in this graph are the gridcells that have a road in them. Given Point1 in the graph with grid coordinates (x_1, y_1) , we define the neighbors of Point1 to be the points in the graph with grid coordinates (x, y) such that $x \in \{x_1 + 1, x_1, x_1 - 1\}$, $y \in \{y_1 - 1, y_1, y_1 + 1\}$ and $(x, y) \neq (x_1, y_1)$ as this would result in Point1 being its own neighbor.³ The weight from Point1 to its neighbors depend on whether they are horizontal, vertical or diagonal neighbors. We assign the weight to be one for the vertical and horizontal neighbors. This is motivated by the fact that our gridcells have the same length and width. While the diagonal neighbors have weight $1.4 \approx \sqrt{2}$.⁴ This is motivated by Pythagoras(right angled triangles) Then we just run the Dijkstra algorithm from every node to the port node. Also, note that we chose the port gridcell to be the gridcell nearest to the port that has all three transportation methods of road, rail and river available.

Let us lastly, explain how we compute the shortest path with two modes of transportation. Let's say we want to go from Point1 to the port using first road and then the river. We compute the shortest path from Point1 to all other points in the road graph with river available and choose the point that lies closest to our original point, Point1. We call this point the transition point. We are interested in the id point and the distance from Point1 to transition point and hence both pieces of information are included as columns in the output CSV file. Finally, we compute the distance from the transition point to the port using river. In our code this information is already available so we just need to access it and no computation of shortest path is actually necessary. In conclusion, we prioritize to get to the second mode of transportation as quickly(in the sense of distance) as possible. This is motivated by the fact that most often the river is the second mode of transportation and we assume that this will be more cost efficient than the first transportation method, especially when that is road or rail.

3 Results

We have applied the methodology described above on Poland. We have chosen the port to in Gdansk. In Figure 1, you can see that we have defined

³We seldom consider ourselves our own neighbors.

⁴This was me being lazy. But this was also motivated by achieving nicer numbers in the resulting CSV file without so many decimals

Poland as our area of interest. The blue line represents the Vistula river, which is the only river we have allowed transportation along.

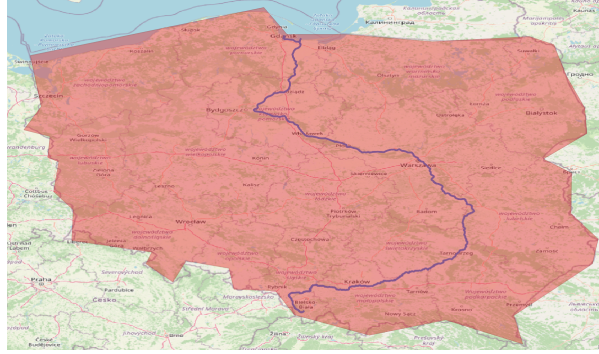


Figure 1: Layout in QGIS before we apply our model.

Below you can see the generated grid after the application of our model.

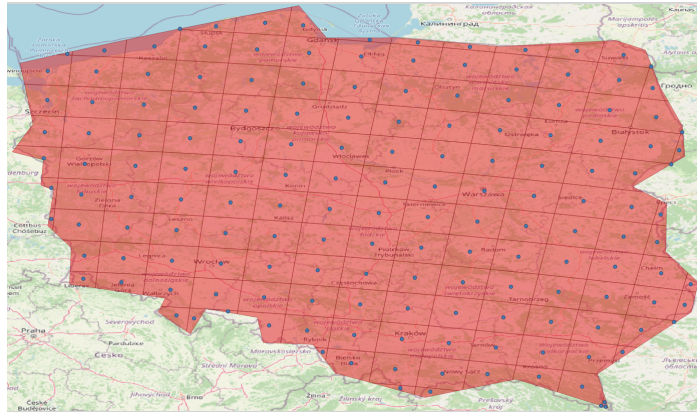


Figure 2

We have also computed the relative error between our algorithm and the shortest path algorithm from QGIS. This resulted in the plot shown below:

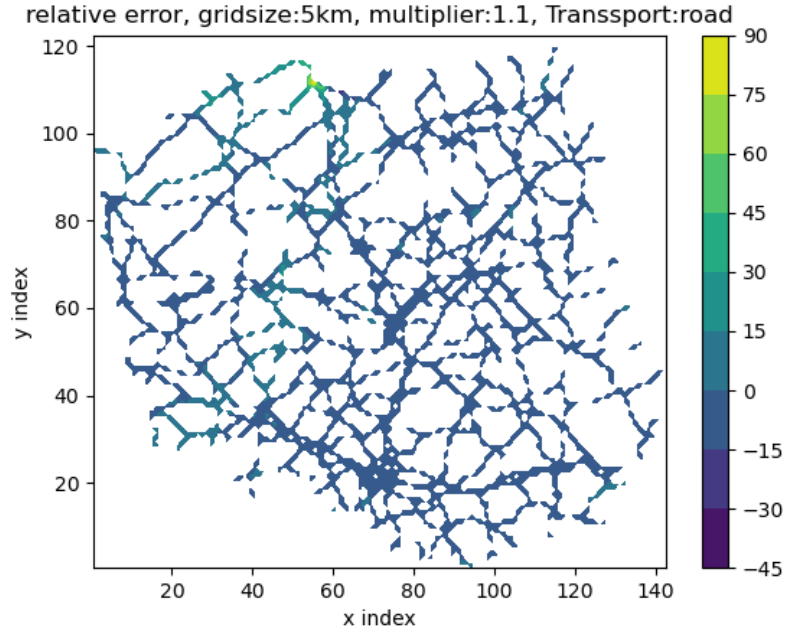


Figure 3: The relative error for a gridsize of 5km, over the road network with multiplier 1.1.

The port is located at around (65,110). The positive values mean that our algorithm is underestimating the distance, while negative values mean that we overestimate the distances.

Also, after testing various various cost multipliers, we have found out that a cost multiplier of 1.0 or 1.1 works best for roads, while for railroads the best choice is 1.1 or 1.2. For rivers these values are 1.3 or 1.4.

4 Discussion

In general it seems like we can get the mean relative error to be below 5% for each of the transportation methods: road, railroad and rivers, if we choose correct multipliers. This analysis should probably be done again if you want to apply this methodology to other regions, as tortuosity may vary and height incline/decline from one place to another may effect the length of the transportation networks as well.

When we run the error analysis in Python, this depends on having some

sort of representation of what the "true/correct" values are. For this we use the QGIS shortest path from layer to point to compute these distances. A couple of things to mention is that this method is not perfect, and contains some mistakes of its own. First of all, the grid points that we use don't lie exactly on the transportation network. This means that these points have to be moved onto the network. This introduces a certain error, as we don't measure the distance of how far these points have to be moved, which in case of bigger grids can lead to some errors. Secondly, the network is not necessarily everywhere connected, i.e. data mistakes. Thus, even if there is actually a road connection in i.e., a junction, the network edges might not connect these and thus the shortest path algorithm will interpret that as there being no road connection there. You can play with tolerance parameters to allow for jumps across such discontinuities, but that is a bit of a hassle. But then the question is how big should the tolerance be, as it also begins to approximate other road segments with linear regression.

Another thing to consider is the use of the QNEAT3 plugin in QGIS to compute distance matrix which computes the distance between all the gridcells, as well as the entry and exit cost, e.g., the distance between points and the closest entry onto the network. This could work as an interesting alternative to the methodology that has been developed using Python. But if you want to try using this, remember to reproject all your layers to the same CRS(the projection of the map). The to consider is: EPSG:3035 : ETRS89-extended/LAEA Europe and EPSG:4258 : ETRS89. In general I liked LAEA Europe better as it uses meters as units, while ETRS89 uses degrees. An interesting observation is that when we ran shortest path in QGIS using two different CRS projections, slightly different results were outputted. I don't know the reason behind this, and somebody with more knowledge in QGIS or map projections should be consulted about what is more accurate.